

Entrega 3 NOSQL Fundamentos de Bases de Datos

Johan Sebastian Castiblanco Galeano – 202316250

Andres Felipe Ramirez Santana - 202311454

Martin De Angulo Ulloa - 202421628

LINK DE API DE RENDER: <https://entrega-nosql.onrender.com>

1a. Análisis de carga de trabajo

Este análisis parte del esquema relacional Oracle ya implementado y de los datos de población provistos, para estimar volúmenes y caracterizar las operaciones del nuevo módulo de reseñas en MongoDB.

1a.i — Cuantificación de entidades

Los siguientes conteos se obtienen del script de población (SQL). Las entidades MongoDB corresponden a proyecciones del enunciado.

Entidad	BD	Registros	Justificación / Proyección
Ciudades	Oracle	8	Catálogo estático, escala mínima.
Hoteles	Oracle	18	2–3 sucursales por ciudad. Estable.
Tipos_habitacion	Oracle	72 (4×18)	4 tipos fijos por hotel.
Habitaciones	Oracle	216 ($3 \times 4 \times 18$)	3 habitaciones por tipo-hotel.
Camas	Oracle	4	Catálogo: King, Queen, Doble, Single.
Camas_por_habitacion	Oracle	126	~1.75 tipos de cama/tipo-habitación.
Comodidades	Oracle	6	Catálogo de amenidades.
Comodidades_por_tipo	Oracle	342	~4.75 comodidades/tipo-habitación.
Servicios	Oracle	11	Catálogo. Crece lentamente.
Servicios_por_tipo	Oracle	468	~6.5 servicios/tipo-habitación.
Usuarios	Oracle	322 actuales → 50 000+	Script: 2 admin + 320 clientes. Real: >50 000.
Administradores	Oracle	2 actuales → ~18–30	Al menos 1 admin por hotel en producción.
Clientes	Oracle	320 actuales → 50 000+	Subconjunto de Usuarios. Crecimiento continuo.
Reservas	Oracle	360 (20×18)	Cada reserva completada es reseña potencial.
Reseñas	MongoDB	~250 000 en 2 años	300–500 nuevas/día. Entidad principal MongoDB.

Entidad	BD	Registros	Justificación / Proyección
Votos de utilidad	MongoDB	~500/día → ~365 000/año	Embebidos en Reseñas. Alta frec. de escritura.
Respuestas admin	MongoDB	~60 % de reseñas (~150 000)	200–300 escrituras/ediciones diarias.

1a.ii — Análisis de operaciones de lectura y escritura

La siguiente tabla identifica las operaciones por entidad, la información que se accede conjuntamente y el tipo de operación.

Entidades	Operación	Información necesaria	Tipo
Reseñas, Hoteles	Consultar reseñas de un hotel paginadas (RF4)	Reseñas + calificación + votos útiles + respuesta admin	Lectura
Reseñas, Reservas, Clientes	Crear reseña (RF1)	Validar reserva completada + datos de la reseña	Escritura
Reseñas	Editar texto y calificación (RF2)	Texto + calificación actualizados	Escritura
Reseñas	Eliminar reseña — cliente (RF3)	ID reseña + ID cliente (soft delete, cambio de estado)	Escritura
Reseñas, Usuarios	Marcar reseña como útil (RF5)	Contador de votos + ID de usuario votante	Escritura
Reseñas, Hoteles	Consultar historial de reseñas propias (RF6)	Reseñas del cliente + estado + calificación + votos + respuesta	Lectura
Reseñas	Responder / editar respuesta admin (RF7)	Texto de respuesta + timestamp + ID admin	Escritura
Reseñas	Eliminar reseña — administrador (RF8)	Estado de reseña + razón de moderación	Escritura
Reseñas, Hoteles	Destacar reseña (RF9)	Flag destacada + desmarcar anterior destacada del mismo hotel	Escritura
Reseñas, Hoteles	Top 10 hoteles por calificación (RFC1)	Calificación promedio + conteo por hotel y período definido	Lectura
Reseñas, Hoteles	Evolución de reputación mes a mes (RFC2)	Promedio mensual de calificación por hotel y año	Lectura
Reseñas, Hoteles, Ciudades	Perfil comparativo hoteles por ciudad (RFC3)	Avg calificación + total reseñas + % con respuesta + % destacadas	Lectura

1a.iii — Cuantificación de operaciones con tasas

Las tasas se derivan directamente de los datos de carga de trabajo provistos en el enunciado.

Entidades	Operación	Información necesaria	Tipo	Tasa estimada
Reseñas, Hoteles	Consultar reseñas de hotel (RF4)	Reseñas + votos + respuesta admin	Lectura	10 000/día (~7/min)

Entidades	Operación	Información necesaria	Tipo	Tasa estimada
Reseñas, Usuarios	Marcar como útil (RF5)	Votos + ID usuario	Escritura	500/día
Reseñas, Reservas	Crear reseña (RF1)	Datos reseña + validación reserva	Escritura	300–500/día
Reseñas	Responder / editar respuesta (RF7)	Texto respuesta admin	Escritura	200–300/día
Reseñas, Hoteles	Consultar historial propio (RF6)	Reseñas por cliente + estado	Lectura	~200/día
Reseñas	Editar reseña (RF2)	Texto + calificación	Escritura	~100/día
Reseñas	Eliminar reseña — cliente (RF3)	Estado de reseña	Escritura	~20/día
Reseñas	Eliminar reseña — admin (RF8)	Estado + razón moderación	Escritura	~10/día
Reseñas, Hoteles	Top 10 por calificación (RFC1)	Avg calificación + período	Lectura	~10/día
Reseñas, Hoteles	Evolución reputación (RFC2)	Promedio mensual por hotel	Lectura	~10/día
Reseñas, Hoteles, Ciudades	Perfil comparativo por ciudad (RFC3)	Métricas comparativas hoteles	Lectura	~10/día
Reseñas, Hoteles	Destacar reseña (RF9)	Flag destacada por hotel	Escritura	~5/día

1b. Modelo conceptual NoSQL — MongoDB

1b.i — Lista de entidades y descripción

El módulo opera sobre las siguientes entidades. Las marcadas como Oracle son fuente en la BD relacional; MongoDB solo guarda su identificador como referencia.

Entidad	Reside en	Descripción
Reseñas	MongoDB	Entidad principal del módulo. Documenta la experiencia de un cliente en una estadia completada. Contiene la calificación numérica (1–5), el texto libre, la fecha de creación, el estado (publicada / eliminada), un indicador de destaque y las sub-estructuras embebidas de respuesta admin y votos. Es la única colección nueva de MongoDB y referencia a Hoteles, Clientes y Reservas de Oracle mediante sus identificadores numéricos.
Respuesta admin	MongoDB (embebida)	Sub-documento embebido dentro de Reseña. Representa la respuesta pública del administrador del hotel a una reseña. Contiene el ID del administrador, el texto de respuesta y la fecha. Puede ser null si el hotel aún no ha respondido, o ser reemplazado cuando el admin edita (RF7). Su existencia depende completamente de la reseña padre.
Votos de utilidad	MongoDB (embebida)	Sub-documento embebido dentro de Reseña. Registra cuántos usuarios autenticados han marcado la reseña como útil (RF5). Almacena un contador (count) y el arreglo de IDs de usuarios votantes, necesario para impedir el doble voto. El crecimiento promedio es bajo (~1.46 votos/reseña/año), por lo que no representa un riesgo de document growth ilimitado.
Hoteles	Oracle	Entidad existente en Oracle. Representa un establecimiento hotelero de la cadena Dann-Alpes. En MongoDB se referencia solo por su id_hotel (NUMBER en Oracle). Se usa para agrupar reseñas y ejecutar las consultas analíticas RFC1–RFC3. El nombre y la ciudad del hotel se obtienen mediante join con Oracle cuando se necesitan para la presentación.
Clientes / Usuarios	Oracle	Entidad existente en Oracle. Representa al huésped que escribe la reseña. En MongoDB se referencia por id_cliente (id_usuario en Oracle). Se valida su existencia y su reserva completada antes de permitir la creación de una reseña (RF1).
Reservas	Oracle	Entidad existente en Oracle. Representa la estadia del cliente. Una reserva con estado 'completada' es la condición necesaria para crear una reseña (RF1). En MongoDB se almacena id_reserva para garantizar la integridad (un cliente no puede reseñar dos veces la misma reserva) y para la trazabilidad.

1b.ii — Relaciones entre entidades y cardinalidad

Entidad A	Entidad B	Cardinalidad	Descripción
Hotel	Reseña	1 : N	Un hotel tiene cero o muchas reseñas. Una reseña pertenece a exactamente un hotel. La referencia id_hotel en cada documento Reseña implementa esta relación.
Cliente	Reseña	1 : N	Un cliente puede escribir múltiples reseñas (una por reserva completada). Una reseña tiene exactamente un autor. La referencia id_cliente en cada documento Reseña implementa esta relación.

Entidad A	Entidad B	Cardinalidad	Descripción
Reserva	Reseña	1 : 1	Una reserva completada puede dar origen a máximo una reseña. Una reseña está asociada a exactamente una reserva (restricción de negocio RF1). La referencia id_reserva garantiza esta unicidad.
Reseña	Respuesta admin	1 : 0..1	Una reseña puede tener cero o una respuesta de administrador. La respuesta no existe sin la reseña. Se modela como sub-documento embebido dentro del documento Reseña.
Reseña	Votos utilidad	1 : N	Una reseña puede acumular múltiples votos de usuarios autenticados. Un usuario puede votar como útil una misma reseña solo una vez (RF5). Se modela como sub-documento embebido {count, usuarios:[]} dentro de Reseña.
Admin	Reseña	1 : N	Un administrador puede responder o eliminar múltiples reseñas del hotel que administra. Esta relación se gestiona mediante el id_admin embebido en la respuesta_admin de cada Reseña.

1b.iii — Análisis de esquema de asociación

Solo se analizan las relaciones en las que interviene al menos una entidad nativa de MongoDB (las referencias a Oracle son siempre referencias por ID por diseño híbrido).

Relación 1 — Reseña ↔ Respuesta del administrador (1:0..1)

Pauta	Pregunta	Emb.	Ref.	Decisión
Simplicity	¿Mantener juntos los datos simplificaría el modelo y el código?	Sí	No	Embeber: un único documento, sin joins.
Go Together	¿Los datos tienen una relación 'contiene' o 'tiene-un'?	Sí	No	Embeber: la respuesta es parte de la reseña.
Query Atomicity	¿La aplicación consulta los datos juntos? (RF4, RF6, RFC1–3)	Sí	No	Embeber: RF4 devuelve reseña + respuesta en bloque.
Update Complexity	¿Se actualizan juntos normalmente?	Sí	No	Embeber: RF7 edita solo el sub-doc respuesta_admin.
Archival	¿Se archivan al mismo tiempo?	Sí	No	Embeber: si la reseña se elimina, la respuesta también.
Cardinality	¿Existe alta cardinalidad en el hijo? (0 ó 1 respuesta por reseña)	Sí	No	Embeber: cardinalidad máxima 1, no hay crecimiento.
Data Duplication	¿La duplicación de datos sería problemática?	Sí	No	Embeber: sin datos duplicados en este caso.
Document Size	¿El tamaño combinado sería excesivo para transferencia?	Sí	No	Embeber: texto corto (~500 caracteres máx.).
Document Growth	¿El sub-documento crece sin límite?	Sí	No	Embeber: máximo 1 respuesta por reseña.

Pauta	Pregunta	Emb.	Ref.	Decisión
Workload	¿Son escrituras independientes frecuentes en workload pesado? (200–300/día)	Sí	No	Embeber: 200–300/día es moderado vs. 10 000 lecturas.
Individuality	¿Puede la respuesta existir sin la reseña padre?	Sí	No	Embeber: la respuesta no tiene significado sin la reseña.

Decisión final: EMBEBER - Respuesta_admin como sub-documento dentro de Reseña.

Relación 2 — Reseña ↔ Votos de utilidad (1:N)

Pauta	Pregunta	Emb.	Ref.	Decisión
Simplicity	¿Mantener juntos simplificaría el modelo?	Sí	No	Embeber: un documento con contador y arreglo.
Go Together	¿Tienen relación 'has-a'?	Sí	No	Embeber: los votos pertenecen a la reseña.
Query Atomicity	¿Se consultan juntos? RF4 muestra count de votos con cada reseña.	Sí	No	Embeber: count embebido evita join para RF4.
Update Complexity	¿Se actualizan juntos? (500 votos/día independientes de la reseña)	No	Sí	Relativo: cada voto es \$inc sobre el documento.
Archival	¿Se archivan al mismo tiempo?	Sí	No	Embeber: los votos carecen de sentido sin la reseña.
Cardinality	¿Alta cardinalidad? (~1.46 votos/reseña/año en promedio)	Sí	No	Embeber: cardinalidad muy baja en promedio.
Data Duplication	¿La duplicación sería problemática?	Sí	No	Embeber: sin datos duplicados.
Document Size	¿El arreglo de voters_ids haría el documento excesivamente grande?	Sí	No	Embeber: tamaño esperado mínimo.
Document Growth	¿El arreglo crece sin límite? (500 votos/día para 250k reseñas = ~0.002/día)	Sí	No	Embeber: crecimiento despreciable por reseña.
Workload	¿Son escrituras ligeras e independientes? (500/día total)	No	Sí	Relativo: \$inc es operación atómica eficiente.
Individuality	¿Puede un voto existir sin la reseña?	Sí	No	Embeber: el voto no existe sin la reseña.

Decisión final: EMBEBER — votos_util como sub-documento {count, usuarios:[]} dentro de Reseña. El operador \$inc actualiza el contador atómicamente; el arreglo usuarios previene el doble voto.

Relación 3 — Hotel (Oracle) ↔ Reseñas (MongoDB) (1:N)

Pauta	Pregunta	Emb.	Ref.	Decisión
Simplicity	¿Embeber los datos del hotel en cada reseña simplificaría el modelo?	No	Sí	Referenciar: Oracle es la fuente de verdad de hoteles.
Go Together	¿La reseña 'contiene' al hotel?	No	Sí	Referenciar: hotel y reseña son entidades independientes.

Pauta	Pregunta	Emb.	Ref.	Decisión
Query Atomicity	¿RFC1–3 necesitan datos del hotel junto con la reseña?	No	Sí	Referenciar: solo id_hotel es suficiente para la agrupación.
Update Complexity	¿Si cambia el nombre del hotel habría que actualizar todas las reseñas?	No	Sí	Referenciar: evitar propagación masiva de cambios.
Archival	¿Las reseñas y el hotel se archivan juntos?	No	Sí	Referenciar: hotel persiste aunque se eliminen reseñas.
Cardinality	¿Alta cardinalidad? (hasta ~13 888 reseñas/hotel en 2 años)	No	Sí	Referenciar: no embeber 13k documentos en uno.
Data Duplication	¿Embeber datos del hotel duplicaría información de Oracle?	No	Sí	Referenciar: evitar inconsistencia entre BD Oracle y MongoDB.
Document Size	¿Embeber datos del hotel haría los documentos excesivamente grandes?	No	Sí	Referenciar: innecesario, solo se necesita el ID.
Document Growth	N/A para esta relación.	No	Sí	Referenciar: el hotel no crece dentro de la reseña.
Workload	¿El hotel se escribe independientemente?	No	Sí	Referenciar: hotel gestionado exclusivamente por Oracle.
Individuality	¿Puede el hotel existir sin reseñas?	No	Sí	Referenciar: el hotel es entidad independiente.

Decisión final: REFERENCIAR - Se almacena id_hotel (NUMBER de Oracle) como campo de referencia en cada documento Reseña.

Relación 4 — Cliente y Reserva (Oracle) ↔ Reseña (MongoDB)

Esta relación sigue el mismo análisis que la Relación 3: Cliente y Reserva son entidades propietarias de Oracle, actualizadas exclusivamente por la BD relacional. En MongoDB se almacenan solo id_cliente e id_reserva como claves de referencia. La decisión es REFERENCIAR por las mismas razones: independencia de ciclo de vida, Oracle como fuente de verdad, sin riesgo de duplicación ni desincronización.

1b.iv — Descripción gráfica con JSON (esquemas de asociación)

Se presentan ejemplos de documentos reales para ilustrar las decisiones de diseño.

Esquema embebido: Reseña con respuesta admin y votos (colección resenas)

La reseña es el documento raíz. respuesta_admin y votos_util son sub-documentos embebidos. Las referencias a Oracle (id_hotel, id_cliente, id_reserva) son campos numéricos simples.

```
// Colección: resenas
{
  "_id"      : ObjectId("64a1b2c3d4e5f6789012abcd"),
  "id_reserva" : 42,           // ref. → Oracle Reservas.id
  "id_hotel"  : 5,            // ref. → Oracle Hoteles.id
  "id_cliente" : 120,         // ref. → Oracle Clientes.id_usuario
  "calificacion" : 4,         // entero 1-5
}
```

```

"texto"      : "Hotel muy cómodo, excelente ubicación cerca del aeropuerto...",
"fecha_creacion": ISODate("2025-03-15T14:30:00Z"),
"estado"     : "publicada",      // "publicada" | "eliminada"
"destacada"  : false,           // solo 1 true por hotel (RF9)

"respuesta_admin": {           // ← EMBEBIDO (1:0..1)
  "id_admin" : 2,
  "texto"    : "Gracias por su visita. Esperamos verle pronto.",
  "fecha"    : ISODate("2025-03-16T09:00:00Z")
},

"votos_util": {               // ← EMBEBIDO (1:N, cardinalidad baja)
  "count"    : 7,              // actualizado con $inc
  "usuarios" : [45, 78, 130, 205, 310, 88, 92] // evita doble voto (RF5)
}
}

```

Esquema referenciado: Hotel (Oracle) ↔ Reseñas (MongoDB)

El hotel vive únicamente en Oracle. MongoDB almacena solo `id_hotel`. Las consultas analíticas (RFC1–RFC3) agrupan por `id_hotel` en MongoDB y luego enriquecen el resultado con los datos del hotel obtenidos de Oracle.

```

// Oracle — tabla Hoteles
| id | nombre                               | id_ciudades | ... |
|----|-----|-----|-----|
| 5 | Dann-Alpes Medellín El Poblado | 2 | ... |

// MongoDB — colección resenas (fragmento de dos documentos del hotel 5)
{
  "_id"      : ObjectId("64a1b2c3d4e5f6789012abcd"),
  "id_hotel" : 5,    // ← referencia al id de Oracle Hoteles
  "id_cliente": 120,
  "id_reserva": 42,
  "calificacion": 4,
  ...
}
{
  "_id"      : ObjectId("64a1b2c3d4e5f6789012ef01"),
  "id_hotel" : 5,    // ← misma referencia
  "id_cliente": 87,
  "id_reserva": 115,
  "calificacion": 5,
  ...
}

// Consulta RFC1: top 10 hoteles — se agrupa por id_hotel en MongoDB,
// luego se cruza con Oracle para obtener nombre y ciudad.
db.resenas.aggregate([
  { $match: { estado: "publicada", fecha_creacion: { $gte: ..., $lte: ... } } },
  { $group: { _id: "$id_hotel", avg_cal: { $avg: "$calificacion" }, total: { $sum: 1 } } },
  { $sort: { avg_cal: -1 } },
  { $limit: 10 }
])

```


Nota: la consistencia entre Oracle y MongoDB se mantiene mediante los identificadores numéricos compartidos (id_hotel, id_cliente, id_reserva).

4. Para todas las consultas se consideraron todas las reseñas, sin importar el estado.

- RFC1: Para esta consulta se usó el documento de reseñas, en primer lugar, se hizo un *match* para definir el periodo, después de filtrar por fecha se hizo un *group* por hoteles donde se crea un nuevo campo que es promedio de calificación a través de *\$avg* y se cuenta el total de reseñas usando *\$sum*, adicionando de a 1, para el tercer *stage* se organiza el promedio de manera descendente para obtener arriba los hoteles con mejor calificación, en seguida se limita a solo 10 valores y por último se proyecta el id del hotel, el promedio de calificación de este redondeado a 2 decimales y el total de reseñas y así obtener los 10 hoteles con mejor calificación promedio en un periodo definido. Para esta consulta se usaron los atributos: *id_hotel*, para filtrar el hotel seleccionado; *fecha_creacion*, para filtrar el periodo con *\$gte* y *\$lte*; *clasificacion*, para calcular el promedio con *\$avg*; e *id_hotel* para agrupar. La consulta utilizada fue:

```
db.resenas.aggregate([
  {
    $match: {
      fecha_creacion: {
        $gte: ISODate("2022-01-01T00:00:00Z"),
        $lte: ISODate("2025-12-31T23:59:59Z")
      }
    },
    {
      $group: {
        _id: "$id_hotel",
        promedio_calificacion: {
          $avg: "$clasificacion"
        },
        total_resenas: {
          $sum: 1
        }
      }
    },
    {
      $sort: {
        promedio_calificacion: -1
      }
    },
    {
      $limit: 10
    }
  ],
```

```
{
  $project: {
    _id: 0,
    hotel_id: "$_id",
    promedio_calificacion: {$round: ["$promedio_calificacion", 2]},
    total_resenas: 1
  }
}
]
```

Donde el periodo se establece en los valores de ISODate en el match, probando desde el 1 de enero de 2022 al 31 de diciembre de 2025 se obtuvieron los siguientes documentos:

promedio_calificacion : 4.8 total_resenas : 5 hotel_id : 7	promedio_calificacion : 4.33 total_resenas : 3 hotel_id : 18
promedio_calificacion : 4.75 total_resenas : 4 hotel_id : 9	promedio_calificacion : 4.33 total_resenas : 3 hotel_id : 12
promedio_calificacion : 4.67 total_resenas : 3 hotel_id : 16	promedio_calificacion : 4.33 total_resenas : 3 hotel_id : 8
promedio_calificacion : 4.67 total_resenas : 3 hotel_id : 3	promedio_calificacion : 4.33 total_resenas : 3 hotel_id : 15
promedio_calificacion : 4.33 total_resenas : 3 hotel_id : 18	promedio_calificacion : 4.25 total_resenas : 8 hotel_id : 4
promedio_calificacion : 4.33 total_resenas : 3 hotel_id : 12	promedio_calificacion : 4.2 total_resenas : 5 hotel_id : 1

- RFC2: Para esta consulta también se usó el documento de reseñas, en primer lugar, se hizo un *match* para definir el año, después se hizo un *group* por año y por mes donde se establece el promedio de calificación a través de *\$avg*, para el tercer *stage* se organiza el mes de manera creciente para obtener los meses en orden y por último se proyecta el id del hotel seleccionado, el año seleccionado, el mes usando *\$ArrayElemAt* para que muestre el nombre del mes según su posición en el array, por eso el primer elemento es " " para que cada mes si equivalga a su número, y el promedio de calificación de cada uno. Para esta consulta se usaron los atributos *id_hotel*, para filtrar el hotel

seleccionado; *fecha_creacion*, para extraer el año con *\$expr* y el mes con *\$month*; y *calificacion* para calcular el promedio con *\$avg*. Se escogió agregación porque se necesitaba agrupar documentos, extrayendo mes y año de una fecha, lo que requiere operadores como *\$year* y *\$month* dentro de un pipeline. La consulta utilizada fue:

```
db.resenas.aggregate([
  {
    $match: {
      id_hotel: 1,
      $expr: {
        $eq: [
          {
            $year: "$fecha_creacion"
          },
          2025
        ]
      }
    },
    {
      $group: {
        _id: {
          anio: {
            $year: "$fecha_creacion"
          },
          mes: {
            $month: "$fecha_creacion"
          }
        },
        id_hotel: {
          $first: "$id_hotel"
        },
        promedio_calificacion: {
          $avg: "$calificacion"
        }
      }
    },
    {
      $sort: {
        "_id.mes": 1
      }
    },
    {
      $project: {
```

```

    _id: 0,
    hotel_id: "$id_hotel",
    anio: "$_id.anio",
    mes: {
      $arrayElemAt: [
        [
          "",
          "Enero",
          "Febrero",
          "Marzo",
          "Abril",
          "Mayo",
          "Junio",
          "Julio",
          "Agosto",
          "Septiembre",
          "Octubre",
          "Noviembre",
          "Diciembre"
        ],
        "$_id.mes"
      ]
    },
    promedio_calificacion: {
      $round: ["$promedio_calificacion", 2]
    },
  }
}
})

```

Donde el id del hotel se define en el *match* en *id_hotel* y el año se establece en los valores de *\$eq* en el *match*, en este ejemplo se usó el hotel de id 1 durante 2025 y se obtuvieron los siguientes documentos:

```
hotel_id : 1
anio : 2025
mes : "Febrero"
promedio_calificacion : 5
```

```
hotel_id : 1
anio : 2025
mes : "Abril"
promedio_calificacion : 4
```

```
hotel_id : 1
anio : 2025
mes : "Julio"
promedio_calificacion : 5
```

```
hotel_id : 1
anio : 2025
mes : "Septiembre"
promedio_calificacion : 4
```

```
hotel_id : 1
anio : 2025
mes : "Noviembre"
promedio_calificacion : 3
```

- RFC3: Para esta consulta se utilizó el documento de reseñas, en primer lugar, se hizo un *match* para definir la ciudad, para esta consulta en especial se agregó a la base de datos la ciudad del hotel según el id para poder hacer la consulta; después se hizo un *group* por el id del hotel donde se establece la ciudad, el promedio de calificación a través de *\$avg*, el total de reseñas usando *\$sum* de 1 en 1, se definen las reseñas con respuesta del administrador contando los documentos donde *respuesta_admin* existe y no es null usando *\$sum*, *\$cond* y *\$ifNull* y el total de reseñas destacadas contando los documentos donde *destacada* es *true* usando *\$sum*, *\$cond* y *\$eq*. Y por último se proyecta el id del hotel seleccionado, la ciudad seleccionada, el promedio de calificación, el porcentaje de reseñas con respuesta y el porcentaje de reseñas destacadas. Para esta consulta se utilizaron los atributos: *ciudad*, para el filtro; *id_hotel*, para agrupar; *calificación*, para el promedio; *respuesta_admin*, para contar reseñas con respuesta usando *\$ifNull* y *\$ne*; y *destacada*, para contar reseñas destacadas con *\$eq*. Se escogió agregación porque se necesitaba calcular múltiples métricas por hotel como promedio, conteos y porcentajes, lo que requiere un pipeline con *\$group* y operadores condicionales como *\$cond*. La consulta utilizada fue:

```
db.resenas.aggregate([
  {
    $match: {
      ciudad: "Cali"
    }
  },
  {
    $group: {
      _id: "$id_hotel",
      ciudad: {
        $first: "$ciudad"
      }
    }
  }
])
```

```

    },
    promedio_calificacion: {
      $avg: "$calificacion"
    },
    total_resenas: {
      $sum: 1
    },
    con_respuesta: {
      $sum: {
        $cond: [
          {
            $and: [
              {
                $ifNull: [
                  "$respuesta_admin",
                  false
                ]
              },
              {
                $ne: ["$respuesta_admin", null]
              }
            ]
          },
          1,
          0
        ]
      }
    },
    destacadas: {
      $sum: {
        $cond: [
          {
            $eq: ["$destacada", true]
          },
          1,
          0
        ]
      }
    }
  },
  {

```

```

$project: {
  _id: 0,
  id_hotel: "$_id",
  ciudad: 1,
  promedio_calificacion: {
    $round: ["$promedio_calificacion", 2]
  },
  total_resenas: 1,
  porcentaje_con_respuesta: {
    $round: [
      {
        $multiply: [
          {
            $divide: [
              "$con_respuesta",
              "$total_resenas"
            ]
          },
          100
        ]
      },
      2
    ]
  },
  porcentaje_destacadas: {
    $round: [
      {
        $multiply: [
          {
            $divide: [
              "$destacadas",
              "$total_resenas"
            ]
          },
          100
        ]
      },
      2
    ]
  }
}

```

]

Donde la ciudad se define en el *match*, en este ejemplo se usó la ciudad de Cali y se obtuvieron los siguientes documentos:

```
ciudad : "Cali"  
total_resenas : 3  
id_hotel : 12  
promedio_calificacion : 4.33  
porcentaje_con_respuesta : 66.67  
porcentaje_destacadas : 33.33
```

```
ciudad : "Cali"  
total_resenas : 3  
id_hotel : 13  
promedio_calificacion : 3.67  
porcentaje_con_respuesta : 33.33  
porcentaje_destacadas : 0
```

5.

a. Implementación de requerimientos funcionales, estos en totales son 9

1. Crear reseña (cliente)
2. Editar reseña (cliente)
3. Eliminar reseña (cliente)
4. Consultar reseñas de un hotel (sin requisitos)
5. Marcar reseña como útil (sin requisitos)
6. Consultar historial de reseñas propias (cliente)
7. Responder reseña (administrador)
8. Eliminar reseña (administrador)
9. Destacar reseña (administrador)

Estas funciones son administradas tanto por clientes, invitados y administradores siendo como se mostró anteriormente en los requisitos, esto se hace de esta forma para delegar funciones específicas a cada rol, aunque cabe recalcar que algunas veces se comparten funciones como el caso de eliminar reseñas.

Para la implementación de apex, debido a que estamos usando render para conectar la base de datos de mongoDB junto a apex, bajo esta premisa se puede intuir que es necesario implementar tanto sql como HTTP, esto se va a evidenciar a medida que avancemos en el análisis de código implementado

1. Creación de reseñas

```
var apiUrl = 'https://entrega-nosql.onrender.com';
```

```
var idReserva = apex.item('P21_ID_RESERVA').getValue();
```

```
var idHotel = apex.item('P21_ID_HOTEL').getValue();
```

```
var idCliente = apex.item('P21_ID_CLIENTE').getValue();
```

```
var califica = apex.item('P21_CALIFICACION').getValue();
```

```
var texto = apex.item('P21_TEXTO').getValue();
```



```

if (!califica || !texto.trim()) {
  apex.message.showErrors([ {
    type: 'error',
    location: 'page',
    message: 'Debes ingresar una calificación y un texto.'
  }]);
  return;
}

fetch(apiUrl + '/resenas', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    id_reserva: parseInt(idReserva),
    id_hotel: parseInt(idHotel),
    id_cliente: parseInt(idCliente),
    calificacion: parseInt(califica),
    texto: texto.trim()
  })
})
.then(function(r) { return r.json(); })
.then(function(data) {
  if (data.id) {
    apex.message.showPageSuccess('¡Reseña publicada exitosamente!');
  } else {
    apex.message.showErrors([ {
      type: 'error',
      location: 'page',
      message: data.detail || 'Error al publicar la reseña.'
    }]);
  }
})
.catch(function() {
  apex.message.showErrors([ {
    type: 'error',
    location: 'page',
    message: 'No se pudo conectar con el servidor.'
  }]);
});

```

En este caso se puede observar que esta conta de var, lo que define donde se guardan las variables siendo cada estas usadas en el resto del código, siendo la más notable la que guarda la URL de la dirección de donde se encuentra toda la información.

Y la información necesaria para publicar de forma correcta las reseñas después se crean las condiciones para verificar si se encuentra el hotel, el cliente, etc. Después se elige como postear que tipos de datos reciben y si estos fueron exitosos o fallidos, junto a si se encontró el servidor, esto último es para validar la correcta conexión entre apex y mongodb. A partir de esto se crean los botones, los ítems ocultos y los textareas, para asegurar el correcto funcionamiento de las hojas en apex. Adicionalmente este código se implementa tanto en los dynamic buttons, (“Se ejecuta cuando ocurre un evento (clic, cambio, etc.)”) Function and Global Variable Declaration (“Se carga primero, antes que todo. Ideal para definir funciones y variables globales”), Execute when Page Loads (“Se ejecuta cuando la página termina de cargar. Aquí llamas las funciones que definiste arriba”)

A continuación, se observarán los resultados, junto a la contraparte representada lo mostrado en mongo

1. crear reseña

Calificación

Dinos tu experiencia

Publicar Reseña

Para la creación de reseñas es necesario una reserva completa y hay que se pueden hacer reseñas únicamente los clientes

Reservas

Nueva reserva

ID	Hotel	Tipo Habitación	Fecha Entrada	Fecha Salida	Num Adultos	Num Niños	Precio Total	Estado	Acción
1	Dann-Alpes Bogota Norte	Habitación Estándar	1/1/2025	1/5/2025	2	2	532000	completada	Reservar
321	Dann-Alpes San Andres Centro	Habitación Estándar	7/10/2025	7/12/2025	2	0	360000	completada	Reservar

El botón donde se puede hacer esto se encuentra en Acción donde es reseñar, en caso de que ya exista una reseña no deja crear, por lo mismo un cliente solo puede tener un comentario por hotel

2. editar reseña

Editar Reseña

Calificación

Tu experiencia

Guardar Cambios

Reseña Actual

Calificación actual: 5/5

Texto actual: Excelente ubicación cerca de la zona empresarial. Habitaciones impecables.

Fecha: 10/2/2025

Votos útiles: 12

En el caso de editar reseñas se puede evidenciar el comentario actual y el resultado de la modificación

Calificación

Tu experiencia

Guardar Cambios

Reseña Actual

Calificación actual: 1/5

Texto actual: zonas sucias

Fecha: 10/2/2025

Votos útiles: 12

3. Eliminar reseñas

Esta tiene la función de eliminar las reseñas, esta la pueden usar tanto los administradores como los clientes

Eliminar Reseña

Eliminar Reseña

Esto se puede evidenciar a continuación

Historial de Reseñas

Orden							
Historial							
Hotel	Calificacion	Texto	Estado	Votos	Respuesta	Fecha	Acciones
2	5/5	el mejor restaurante del mundo...	eliminada	0	No	26/5/2026	-
15	4/5	Excelente vista al Caribe. Para descansar es perfecto....	eliminada	5	No	5/3/2025	-
1	1/5	zonas sucias...	publicada	12	Si	10/2/2025	Editar Eliminar
7	5/5	Playa privada increíble, atención de primera. Una experienci...	publicada	15	Si	5/2/2025	Editar Eliminar
4	3/5	Buena experiencia inicial. Algunas demoras en check-in....	publicada	2	No	15/1/2025	Editar Eliminar

Siendo esta la de los clientes

Panel Admin Reseñas

Selecciona Hotel

Dann-Alpes Barranquilla Pto Colombia

Reseñas

ID	Calificacion	Texto	Votos	Destacada	Acciones
6a1555e5...	5/5	Mejor de lo esperado. El restaurante de mariscos e...	6	No	Responder Moderar Destacar

4. consulta reseñas

Reseñas del Hotel

Ordenar por

Ordenar por

Reseñas

Selecciona un hotel:

Dann-Alpes Bogota Centro

Resena Destacada

Buen valor por el precio. Personal muy amable.

Calificacion: 4/5 | 8 util

Calificacion: 4/5

Me sorprendió la vista desde el piso 12. Recomendado.

25/10/2025

9 personas lo encontraron útil [Util](#)

Respuesta del hotel: Gracias, las habitaciones altas son muy solicitadas por la vista.

Calificacion: 3/5

Hotel correcto, ubicación céntrica útil pero la zona es ruidosa en la noche.

14/3/2025

5 personas lo encontraron útil [Util](#)

Respuesta del hotel: buena master

En esta ocacion s epueden observar todas las reseñas, todo tipo de usuario con un filtro por hotel.

5. historial de usuario

Historial de Reseñas

Orden

Historial

Hotel	Calificacion	Texto	Estado	Votos	Respuesta	Fecha	Acciones
2	5/5	el mejor restaurante del mundo...	eliminada	0	No	26/5/2026	-
15	4/5	Excelente vista al Caribe. Para descansar es perfecto....	eliminada	5	No	5/3/2025	-
1	1/5	zonas sucias...	publicada	12	Si	10/2/2025	Editar Eliminar
7	5/5	Playa privada increíble, atención de primera. Una experienci...	publicada	15	Si	5/2/2025	Editar Eliminar
4	3/5	Buena experiencia inicial. Algunas demoras en check-in...	publicada	2	No	15/1/2025	Editar Eliminar

Cada cliente puede observar sus reseñas, eliminarlas y modificarlas para eso estn implementados los botones de eliminar y editar

6. destacar reseñas

Destacar Reseña

Destacar Reseña

Reseña a Destacar

Calificacion: 5/5

Texto: Perfecto para festival de salsa. Personal facilitó todo.

Fecha: 5/10/2025

Votos utiles: 8

Actualmente destacada: Si

Respuesta admin: Apoyar la cultura caleña es nuestra prioridad.

Una funcion unica del administrador.

Escenarios de Prueba:

A continuación, se muestran algunos escenarios de prueba según el esquema de validación, probando que documentos que no cumplan con este no pasan.

1.

```
// — TEST 1: Falta campo requerido (texto)
probar("RF1-T1 – Falta campo requerido: 'texto'", () => {
  db.resenas.insertOne({
    id_reserva: 9999,
    id_hotel: 1,
    id_cliente: 3,
    calificacion: 4,
    // texto: ausente intencionalmente
    fecha_creacion: new Date(),
    estado: "publicada",
    destacada: false,
    votos_util: { count: 0, usuarios: [] }
  });
});
```

Resultado:

```
< TEST: RF1-T1 – Falta campo requerido: 'texto'
<  RESULTADO: Documento rechazado correctamente
< Error: Document failed validation
```

2.

```
// — TEST 2: Calificación fuera de rango (> 5)
probar("RF1-T2 – Calificación fuera de rango: calificacion = 6", () => {
  db.resenas.insertOne({
    id_reserva: 9998,
    id_hotel: 1,
    id_cliente: 3,
    calificacion: 6, // máximo permitido es 5
    texto: "Muy buen hotel",
    fecha_creacion: new Date(),
    estado: "publicada",
    destacada: false,
    votos_util: { count: 0, usuarios: [] }
  });
});
```

Resultado:

```
< TEST: RF1-T2 – Calificación fuera de rango: calificacion = 6
<  RESULTADO: Documento rechazado correctamente
< Error: Document failed validation
```

```
// — TEST 5: Texto vacío (minLength = 1)
probar("RF1-T5 – Texto vacío: texto = ''", () => {
  db.resenas.insertOne({
    id_reserva: 9995,
    id_hotel: 1,
    id_cliente: 3,
    calificacion: 3,
    texto: "", // mínimo 1 carácter
    fecha_creacion: new Date(),
    estado: "publicada",
    destacada: false,
    votos_util: { count: 0, usuarios: [] }
  });
});
```

```
< TEST: RF1-T5 – Texto vacío: texto = ''
<  RESULTADO: Documento rechazado correctamente
< Error: Document failed validation
<
```

```
// — TEST 6: votos_util mal formado (falta 'usuarios')
probar("RF1-T6 – votos_util incompleto: falta campo 'usuarios'", () => {
  db.resenas.insertOne({
    id_reserva: 9993,
    id_hotel: 1,
    id_cliente: 3,
    calificacion: 4,
    texto: "Buena experiencia",
    fecha_creacion: new Date(),
    estado: "publicada",
    destacada: false,
    votos_util: { count: 0 } // falta 'usuarios'
  });
});
```

Resultado:

```
< TEST: RF1-T6 – votos_util incompleto: falta campo 'usuarios'
<  RESULTADO: Documento rechazado correctamente
< Error: Document failed validation
<
```



```
// — TEST 7: respuesta_admin mal formada (falta 'fecha')
probar("RF1-T7 – respuesta_admin incompleta: falta campo 'fecha'", () => {
  db.resenas.insertOne({
    id_reserva: 9990,
    id_hotel: 1,
    id_cliente: 3,
    calificacion: 4,
    texto: "Buena experiencia",
    fecha_creacion: new Date(),
    estado: "publicada",
    destacada: false,
    votos_util: { count: 0, usuarios: [] },
    respuesta_admin: {
      id_admin: 1,
      texto: "Gracias por su visita"
      // fecha: ausente intencionalmente
    }
  });
});
```

Resultado:

```
< TEST: RF1-T7 – respuesta_admin incompleta: falta campo 'fecha'
<  RESULTADO: Documento rechazado correctamente
< Error: Document failed validation
<
```

Updates invalidos:

```

// — TEST 8: Update con calificación inválida
probar("RF2-T8 – Editar calificacion a valor inválido: calificacion = 10", () => {
  db.resenas.updateOne(
    { _id: idBase },
    { $set: { calificacion: 10 } } // máximo es 5
  );
});

// — TEST 9: Update con texto vacío
probar("RF2-T9 – Editar texto a string vacío: texto = '', () => {
  db.resenas.updateOne(
    { _id: idBase },
    { $set: { texto: "" } } // minLength es 1
  );
});

// — TEST 10: Update que cambia estado a valor no permitido
probar("RF2-T10 – Editar estado a valor no permitido: estado = 'suspendida', () => {
  db.resenas.updateOne(
    { _id: idBase },
    { $set: { estado: "suspendida" } } // solo "publicada" o "eliminada"
  );
});

```

Resultado:

RF2 – UPDATES INVÁLIDOS

— Insertando reseña base válida para updates...

Reseña base insertada con _id: 6a1a05530153aa1c665cbdcc

TEST: RF2-T8 – Editar calificacion a valor inválido: calificacion = 10

✅ RESULTADO: Documento rechazado correctamente

Error: Document failed validation

TEST: RF2-T9 – Editar texto a string vacío: texto = ''

✅ RESULTADO: Documento rechazado correctamente

Error: Document failed validation

TEST: RF2-T10 – Editar estado a valor no permitido: estado = 'suspendida'

✅ RESULTADO: Documento rechazado correctamente

Error: Document failed validation

Y a continuación unos documentos que si aceptaron su creación (cumplen las reglas de validación).

```

print("\n— Insertando documento válido con respuesta admin...");
try {
  db.resenas.insertOne({
    id_reserva: 8004,
    id_hotel: 5,
    id_cliente: 22,
    calificacion: 4,
    texto: "Muy buena experiencia en general.",
    fecha_creacion: new Date(),
    estado: "publicada",
    destacada: false,
    votos_util: { count: 1, usuarios: [78] },
    respuesta_admin: {
      id_admin: 2,
      texto: "Gracias por elegirnos, esperamos verle pronto.",
      fecha: new Date()
    }
  });
  print("✅ Insertado correctamente (con respuesta_admin)");
} catch (e) {
  print("❌ Error inesperado: " + e.message);
}

print("\n— Update válido sobre reseña base (RF2)...");
try {
  db.resenas.updateOne(
    { _id: idBase },
    { $set: { calificacion: 5, texto: "Actualicé mi reseña, fue mejor de lo que pensé." } }
  );
  print("✅ Update válido aceptado correctamente");
} catch (e) {
  print("❌ Error inesperado: " + e.message);
}

```

Resultados:

```
— Insertando documento válido sin respuesta admin...
✓ Insertado correctamente (sin respuesta_admin)

— Insertando documento válido con respuesta admin...
✓ Insertado correctamente (con respuesta_admin)

— Update válido sobre reseña base (RF2)...
✓ Update válido aceptado correctamente
```

Para ver mejor estos escenarios ver script en el repositorio.